Day 26: Incident Evidence Collection

Evidence Package for INC-20260929-001: Repeated Failed Logons Against CyberAmy

SOC Internship - TriosCyber

Field	Details
Intern Name	Amrita Pathak
Date	29 September 2026
Internship Track	SOC (Security Operations Center)
Assigned Task	Take one suspicious pattern from your lab and collect supporting evidence: affected endpoint, user, timestamps, source details, relevant logs and screenshots. Organize the evidence in a clear incident folder structure. (Tools: Wazuh, endpoint logs)
Status of this Report	**Practical exercise, fully completed. A real incident case folder (INC-20260929-001) was built on the DESKTOP-AUHDFVM lab VM containing exported raw Security logs, a parsed copy, endpoint state captures, screenshots, case notes, a chain of custody record, and SHA-256 hashes of every evidence file. Every screenshot in this report was captured during the exercise. The only element not collected is a Wazuh alert, because no Wazuh manager is deployed in this lab (see Days 17-20); that gap is documented inside the case folder itself.**

Table of Contents

Table of Contents 2
Key Terms 3
Important Note on This Report 3
Executive Summary 4
1. Objective 5
2. Pattern Selected 5
3. Building the Case Folder 5
4. Time Zone Normalization 7
5. Collecting the Logs 8
5.1 Exporting the Raw Security Log 8
5.2 Parsing a Readable Copy 9
6. Findings from the Evidence 14
6.1 Affected Endpoint and User 14
6.2 Timestamps 14
6.3 Source Details 14
6.4 Failure Reason: SubStatus 0xC000006A 15
6.5 Two Successful Logon Events at 13:12:10 15
7. Endpoint Information Collected 17
8. Screenshots and the Wazuh Folder 20
8.1 Contents of 01_Screenshots 20
8.2 The Wazuh Status Note 23
9. Case Notes and Chain of Custody 25
10. Sealing the Evidence 28
11. Handling Issues During Collection 31
12. Evidence Summary 31
13. Conclusion 32
References 32

Key Terms

- **Evidence Package:** A self-contained, organized collection of everything needed for someone who was not involved to independently verify what happened: raw data, readable copies, context, notes, and integrity records.
- **Primary vs. Supporting Evidence:** Primary evidence is what the system itself recorded (here, the exported .evtx log files). Supporting evidence, such as screenshots, shows what the analyst saw on screen but cannot be re-examined or re-filtered the way a log file can.
- **Chain of Custody:** A written record of who collected each item of evidence, when, how, and every time it was moved or handled afterwards, so its integrity can be defended later.
- **SHA-256 Hash:** A 64-character fingerprint calculated from a file's exact contents. Changing even one byte of the file produces a completely different hash, so hashes recorded at collection time prove later whether a file has been altered. SHA-256 is the default algorithm of PowerShell's Get-FileHash cmdlet.
- **Order of Volatility:** The principle in RFC 3227 (Guidelines for Evidence Collection and Archiving, February 2002) that collection should proceed from the most volatile evidence to the least volatile, so the data most likely to disappear is captured first.
- **XPath Query:** The filter language wevtutil uses with its /q option to select which events to export, for example only Event IDs 4624 and 4625 between two UTC times.
- **SubStatus Code:** A secondary code in Event ID 4625 that narrows down why a logon failed. 0xC000006A means the user name is correct but the password is wrong; 0xC0000064 means the specified account does not exist.
- **UAC Split Token (Linked Logon):** When an administrator logs on interactively with User Account Control enabled, Windows creates two logon sessions, one with administrator privilege and one without. Each is logged as its own Event ID 4624, and the Linked Logon ID field in each points to the other.

Important Note on This Report

All evidence in this report is real and was collected from the lab VM during this session. The incident examined is the controlled brute-force style sequence generated in Day 25, treated here as if it were a genuine alert requiring a defensible evidence package. The only element that could not be collected is a Wazuh alert, because no Wazuh manager is deployed in this lab (see Days 17-20). Rather than leave that folder empty, the case contains a written status note explaining the gap and the detection that would be expected.

Executive Summary

Day 26 required turning a suspicious pattern into an organized, defensible body of evidence. The pattern chosen was the Day 25 sequence of eight failed logons against the CyberAmy account, followed by a successful logon. A case folder, INC-20260929-001, was created on the affected endpoint with six numbered subfolders separating case notes, screenshots, raw logs, parsed logs, endpoint information, and Wazuh material.

The Security log was exported twice with wevtutil: once filtered to the six-minute incident window, and once in full. A readable CSV was then parsed from the exported file rather than from the live log. This re-analysis confirmed the original eight failure timestamps exactly and surfaced three details not established in Day 25: SubStatus 0xC000006A shows the attempts used a valid user name; source address 127.0.0.1 with Logon Type 2 places the attempts at the machine's own console; and the successful logon was recorded as two linked 4624 events, proven to be a UAC split token.

Endpoint state was captured into eight text files, six screenshots were placed in the case, and a case summary, chain of custody record, evidence inventory and SHA-256 hash manifest were written. All 18 evidence files were hashed as the final step. Three handling issues arose during collection; each was corrected before sealing and is documented rather than hidden.

1. Objective

Select one suspicious pattern from the lab, collect every piece of evidence needed to describe it (affected endpoint, user, timestamps, source details, logs and screenshots), and organize it into an incident folder that another analyst could pick up, understand, and verify without speaking to the person who collected it.

2. Pattern Selected

The Day 25 sequence was chosen: eight failed interactive logons against CyberAmy between 13:07:33 and 13:11:19 IST, followed by a successful logon at 13:12:10. Three reasons made it the right choice:

- It is real telemetry with known, verified timestamps, not an illustrative example.
- It matches a genuine Wazuh correlation rule (60204: 8 failures from one source within 240 seconds), so it is the kind of pattern a SOC would actually receive as an alert.
- Day 25 decided whether the pattern should be escalated. Day 26 is the next step in a real workflow: building the case that supports that decision.

All commands below were run in Windows PowerShell opened as Administrator on the DESKTOP-AUHDFVM VM. In the code blocks, cmdlets appear in yellow, parameters in blue, variables in light blue, text strings in orange, and built-in Windows programs in teal.

3. Building the Case Folder

Step 1: Create the folder structure

A variable holding the case path was set first, so every later command could refer to $case instead of retyping the full path. The six subfolders were then created in one pipeline:

```
$case = "C:\Incidents\INC-20260929-001"
"00_Case_Notes","01_Screenshots","02_Logs_Raw","03_Logs_Parsed",
"04_Endpoint_Info","05_Wazuh" | ForEach-Object {
    New-Item -ItemType Directory -Path "$case\$_" -Force
}
```

The folder name follows the convention INC-date-sequence, so cases sort by date and never collide. The numeric prefixes keep every case folder in the same order, so any analyst knows where to look.

Step 2: Confirm the time zone

```
tzutil /g
```

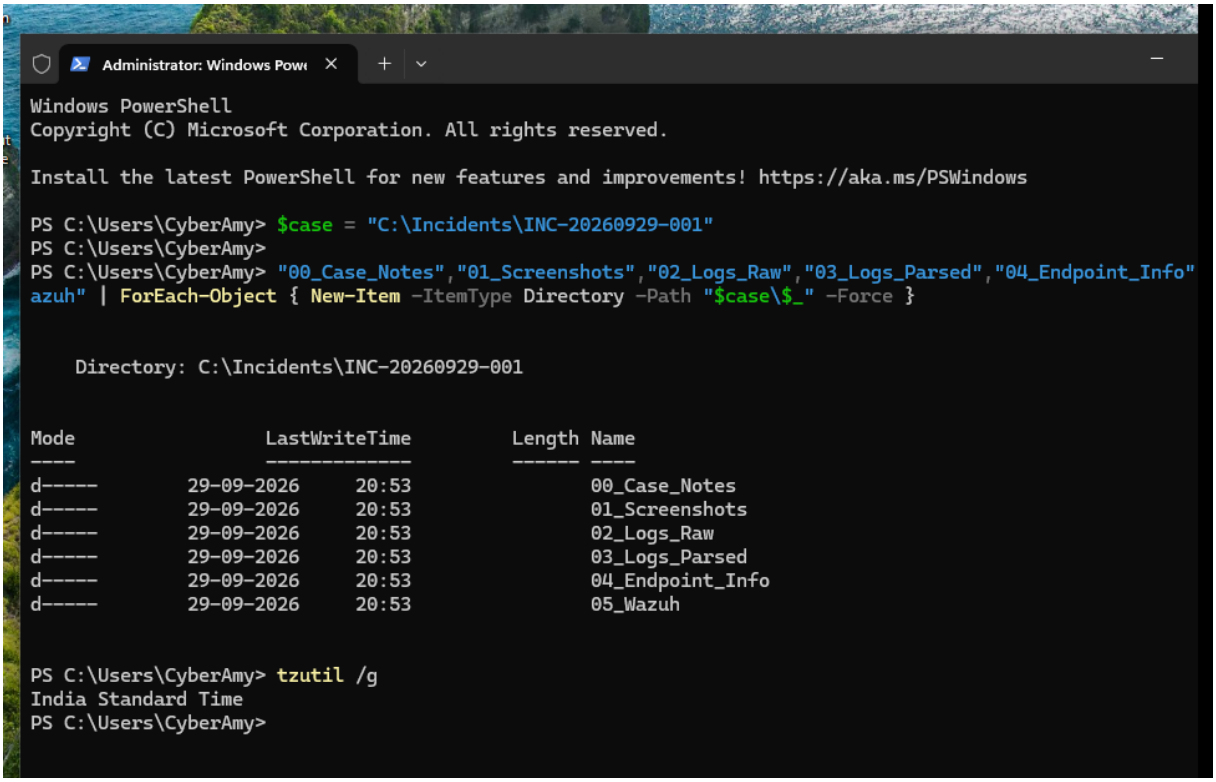


Figure 1. Case folder created, followed by the time zone check.

What this screenshot shows:

- The first line assigns the case path to the variable $case. PowerShell prints nothing for an assignment, which is expected.
- The second command feeds the six folder names into New-Item. The output table lists each new folder, with Mode d----- meaning directory and LastWriteTime 29-09-2026 20:53 showing when they were created.
- tzutil /g prints India Standard Time, which is UTC+05:30. Every timestamp in this case is therefore local IST, 5 hours 30 minutes ahead of UTC.

The role of each folder:

Folder	Contents	Role
00_Case_Notes	Case summary, chain of custody, evidence inventory, hash manifest	Paperwork describing the evidence; not itself hashed
01_Screenshots	SS01 to SS06	Supporting evidence
02_Logs_Raw	Security_INC_window.evtx, Security_full.evtx	Primary evidence
03_Logs_Parsed	INC_window_parsed.csv	Readable working copy
04_Endpoint_Info	8 text captures of system state	Context for the endpoint and account
05_Wazuh	wazuh_status_note.txt	Documents the detection gap

4. Time Zone Normalization

Event Viewer displays local time, but Windows stores each event's TimeCreated SystemTime value in UTC (ISO 8601 format ending in Z). The export query in Section 5 filters on that stored value, so the incident window had to be converted to UTC first. Mixing the two is one of the most common evidence errors in incident work.

Point in the incident	Local time (IST)	UTC
Export window start	13:07:00	07:37:00
First failed logon	13:07:33	07:37:33
Last failed logon	13:11:19	07:41:19
Successful logon	13:12:10	07:42:10
Export window end	13:13:00	07:43:00

5. Collecting the Logs

5.1 Exporting the Raw Security Log

RFC 3227 advises collecting the most volatile evidence first. By default Windows overwrites the oldest event log entries once a log reaches its maximum size, so the Security log entries for this incident would eventually disappear; exporting them was therefore the first collection step. (In a live incident, memory would be captured even earlier, since RFC 3227 ranks it more volatile than disk. Memory capture was not needed here because the activity was a completed, self-run lab test.)

Step 3: Export the incident window and the full log

```
$q = "*[System[(EventID=4624 or EventID=4625) and
  TimeCreated[@SystemTime>='2026-09-29T07:37:00.000Z' and
  @SystemTime<='2026-09-29T07:43:00.000Z']]]"
wevtutil epl Security "$case\02_Logs_Raw\Security_INC_window.evtx" "/q:$q"
wevtutil epl Security "$case\02_Logs_Raw\Security_full.evtx"
```

(The $q query is split across three lines here for readability; it was entered as one line.) How the commands work:

- $q holds an XPath query selecting only Event IDs 4624 (successful logon) and 4625 (failed logon) whose SystemTime falls between 07:37:00 and 07:43:00 UTC, the incident window from Section 4.
- wevtutil epl (export-log) copies events from the named log into a new .evtx file. It reads the log; it does not clear or modify it. The /q option applies the XPath filter.
- The second export has no filter, so it copies the entire Security log. Narrow collection cannot be undone later, but broad collection only costs disk space.

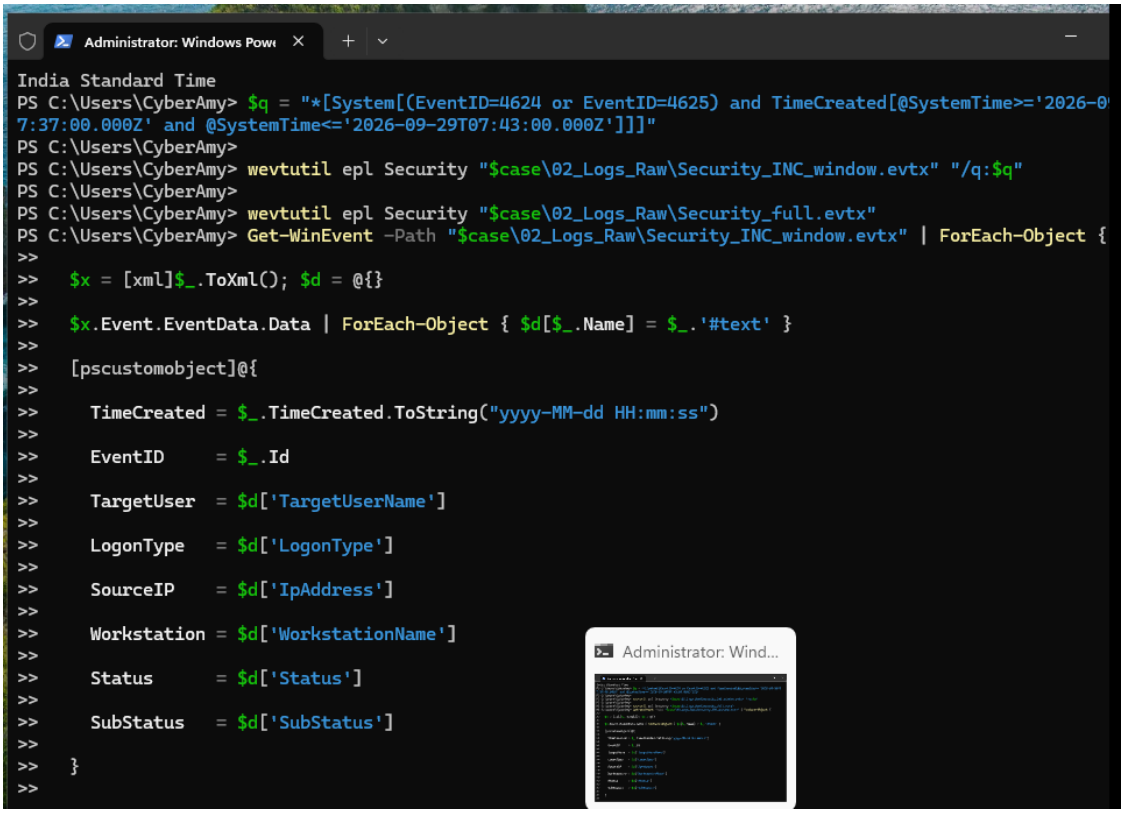


Figure 2. The query definition, both wevtutil exports, and the start of the parsing command.

What this screenshot shows:

- The first three prompts define $q and run both wevtutil exports. Neither export printed anything: for wevtutil, no output means success.
- The next command begins with Get-WinEvent -Path, reading the exported file rather than the live log.
- The lines starting with >> are PowerShell's continuation prompt: it is waiting for the rest of a multi-line command because the opening { has not yet been closed. The blank >> lines between them came from pasting and are harmless.

5.2 Parsing a Readable Copy

Step 4: Parse the exported window into a CSV

Get-WinEvent read each event from the exported file. For each one, the event's XML was loaded and its named EventData fields were copied into a lookup table ($d), from which eight columns were built:

```powershell
$rows = Get-WinEvent -Path "$case\02_Logs_Raw\Security_INC_window.evtx" |
  ForEach-Object {
    $x = [xml]$_.ToXml(); $d = @{}
    $x.Event.EventData.Data | ForEach-Object { $d[$_.Name] = $_.'#text' }
    [pscustomobject]@{
      TimeCreated = $_.TimeCreated.ToString("yyyy-MM-dd HH:mm:ss")
      EventID     = $_.Id
      TargetUser  = $d['TargetUserName']
      LogonType   = $d['LogonType']
      SourceIP    = $d['IpAddress']
      Workstation = $d['WorkstationName']
      Status      = $d['Status']
      SubStatus   = $d['SubStatus']
    }
  } | Sort-Object TimeCreated
$rows | Export-Csv "$case\03_Logs_Parsed\INC_window_parsed.csv" -NoTypeInformation
$rows | Format-Table -AutoSize
```

-NoTypeInformation stops Windows PowerShell 5.1 from writing an extra #TYPE header line at the top of the CSV. The first attempt at this step failed, as shown next.

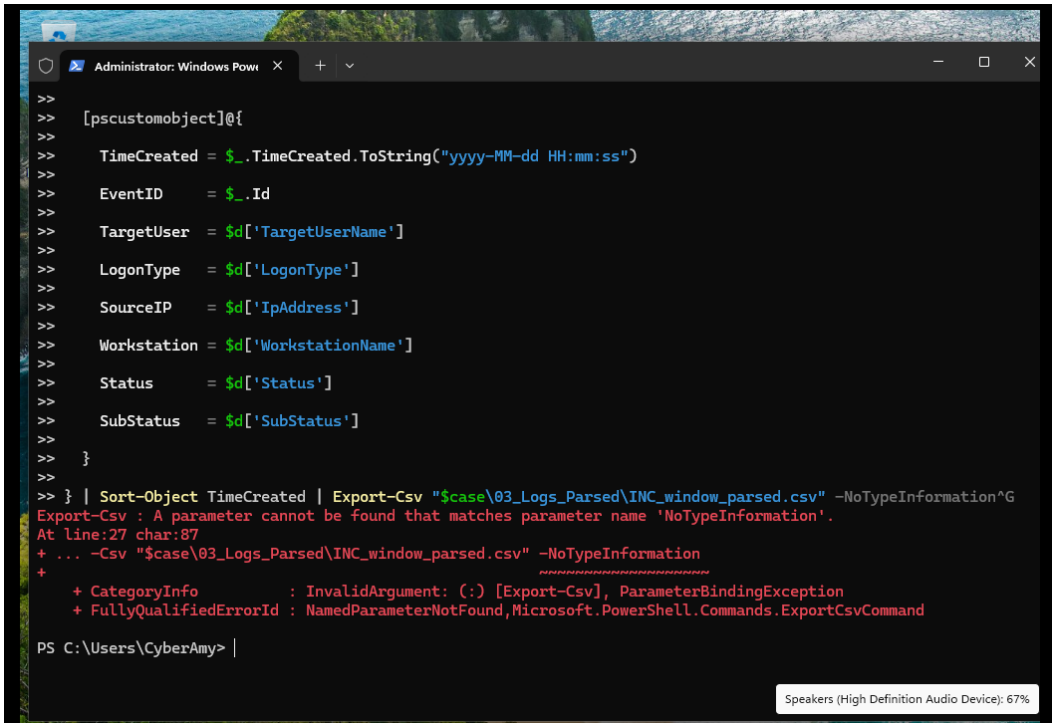


Figure 3. The first parsing attempt, rejected because of a hidden character.

What this screenshot shows:

- The final line of the command ends in -NoTypeInformation^G. The ^G is a hidden control character that was attached to the end of the line when it was pasted into the VM.
- PowerShell reports: A parameter cannot be found that matches parameter name 'NoTypeInformation'. It read the parameter as NoTypeInformation plus the hidden character, which is not a real parameter.
- ParameterBindingException means PowerShell rejected the whole command before running any part of it, so no file was written and nothing needed cleaning up.
- The fix was to split the command, storing the results in $rows first and then typing the short export line by hand, as shown in the Step 4 code above.

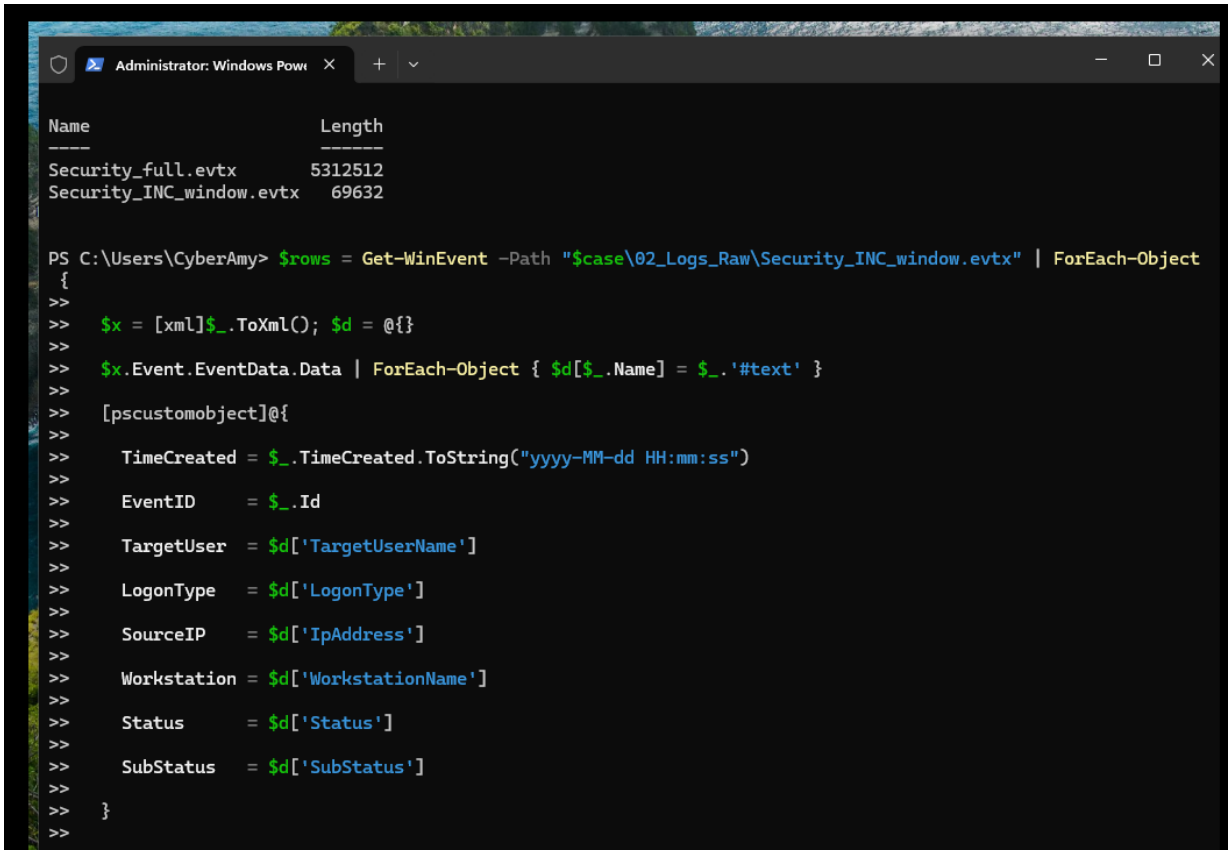


Figure 4. Raw log files confirmed, and the parsing command re-run.

What this screenshot shows:

- The file listing at the top confirms both exports exist: Security_full.evtx at 5,312,512 bytes (the whole Security log) and Security_INC_window.evtx at 69,632 bytes (only the incident window).
- Below it, the parsing command was re-entered, this time assigning its output to $rows instead of piping straight into Export-Csv.

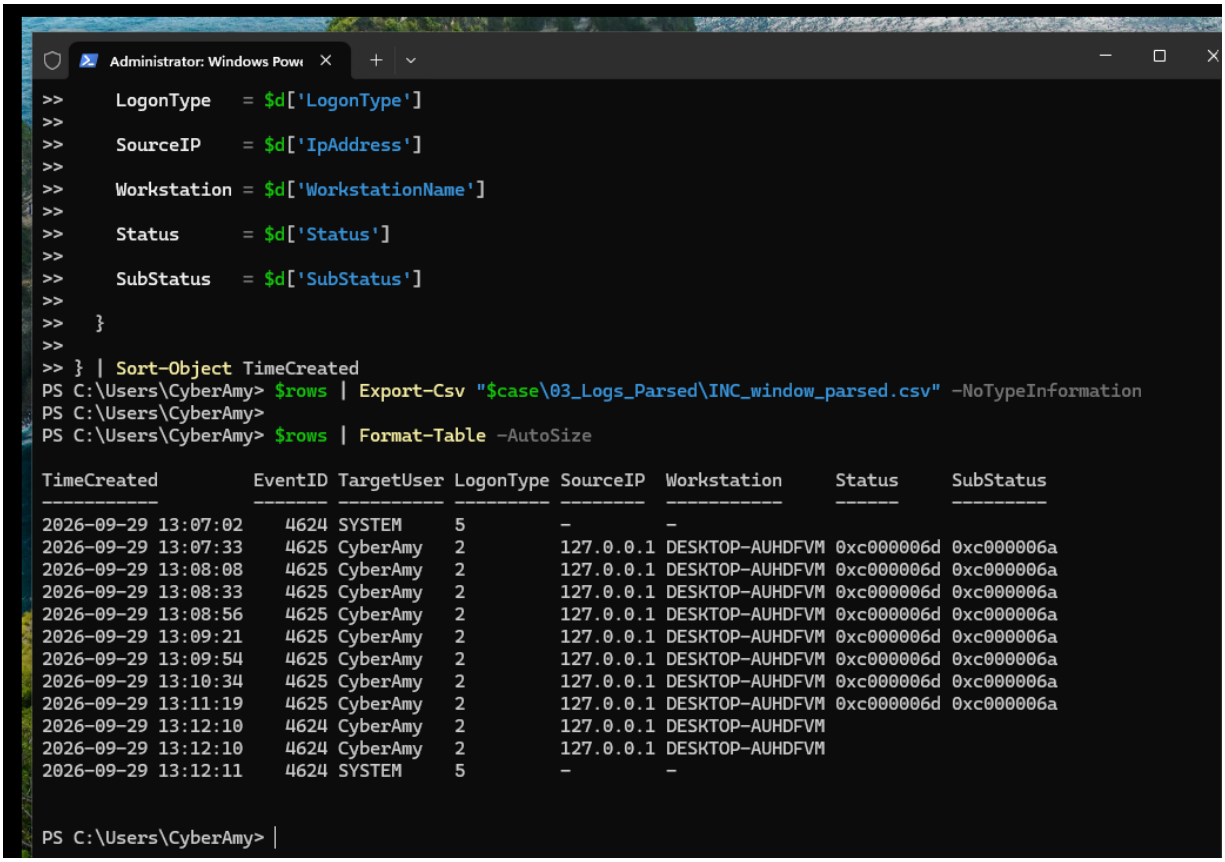


Figure 5. The parsed incident window, saved to CSV and displayed as a table (saved in the case as SS05).

What this screenshot shows:

- The two short commands after the parsing block ran with no errors: Export-Csv wrote the CSV, and Format-Table displayed the same rows on screen.
- Rows 2 to 9 are the eight failed logons (Event ID 4625) for CyberAmy from 13:07:33 to 13:11:19, all Logon Type 2, source IP 127.0.0.1, workstation DESKTOP-AUHDFVM, Status 0xc000006d and SubStatus 0xc000006a.

- The two rows at 13:12:10 are Event ID 4624 for CyberAmy: the successful logon, recorded twice (explained in Section 6.5). Successful logons have no Status or SubStatus values, so those columns are blank.
- The first and last rows are SYSTEM with Logon Type 5 (Service), meaning Windows' Service Control Manager started a service. They are background activity, not part of the incident, and are kept in the export so the evidence shows everything in the window.

6. Findings from the Evidence

6.1 Affected Endpoint and User

Endpoint	DESKTOP-AUHDFVM (Windows VM on VMware)
Time zone	India Standard Time (UTC+05:30)
Affected account	CyberAmy
Account privilege	Member of the local Administrators group (Figure 8)

6.2 Timestamps

Reading the exported evidence file independently reproduced the Day 25 timestamps exactly. Re-deriving findings from the collected evidence, instead of trusting the original on-screen observation, is what makes them defensible.

#	Time (IST)	Event ID	Result
1	13:07:33	4625	Failed
2	13:08:08	4625	Failed
3	13:08:33	4625	Failed
4	13:08:56	4625	Failed
5	13:09:21	4625	Failed
6	13:09:54	4625	Failed
7	13:10:34	4625	Failed
8	13:11:19	4625	Failed
9	13:12:10	4624 (x2, linked)	Successful

6.3 Source Details

All eight failures recorded Logon Type 2, source IP address 127.0.0.1, and workstation DESKTOP-AUHDFVM. Microsoft defines Logon Type 2 (Interactive) as a user logging on to this computer, and documents 127.0.0.1 in the source address field as meaning localhost. Together these show the attempts were typed at the machine's own console, not sent from another computer. A Remote Desktop attempt would instead record Logon Type 10 (RemoteInteractive)

and the connecting machine's address. Because every failure shares the same source address, this also confirms that Wazuh rule 60204's same-source condition, discussed in Day 25, would genuinely have been met.

6.4 Failure Reason: SubStatus 0xC000006A

Every failure recorded Status 0xC000006D, meaning the attempted logon is invalid due to a bad user name or bad authentication information. The SubStatus narrows it down: 0xC000006A means the user name is correct but the password is wrong. Day 21 recorded only the Status code. In a real incident the difference matters: 0xC0000064 (account does not exist) would indicate someone guessing user names, whereas 0xC000006A shows the person already knew a valid account name, which makes the attempt targeted.

6.5 Two Successful Logon Events at 13:12:10

The parsed log showed two CyberAmy 4624 events in the same second. Instead of assuming the cause, the likely explanation (a UAC split token for an administrator account) was stated as a prediction and then tested against the evidence file:

Step 5: Test the linked-logon prediction

```
Get-WinEvent -Path "$case\02_Logs_Raw\Security_INC_window.evtx" |
  Where-Object { $_.Id -eq 4624 -and
    $_.TimeCreated.ToString("HH:mm:ss") -eq "13:12:10" } |
  ForEach-Object {
    $x = [xml]$_.ToXml(); $d = @{}
    $x.Event.EventData.Data | ForEach-Object { $d[$_.Name] = $_.'#text' }
    [pscustomobject]@{ LogonId = $d['TargetLogonId'];
      LinkedLogonId = $d['TargetLinkedLogonId'];
      ElevatedToken = $d['ElevatedToken'] }
  } | Format-Table -AutoSize
```

If the prediction is right, the two events should point to each other through their Linked Logon IDs, and exactly one should hold an elevated token. In the raw event data, %%1842 means Yes and %%1843 means No.

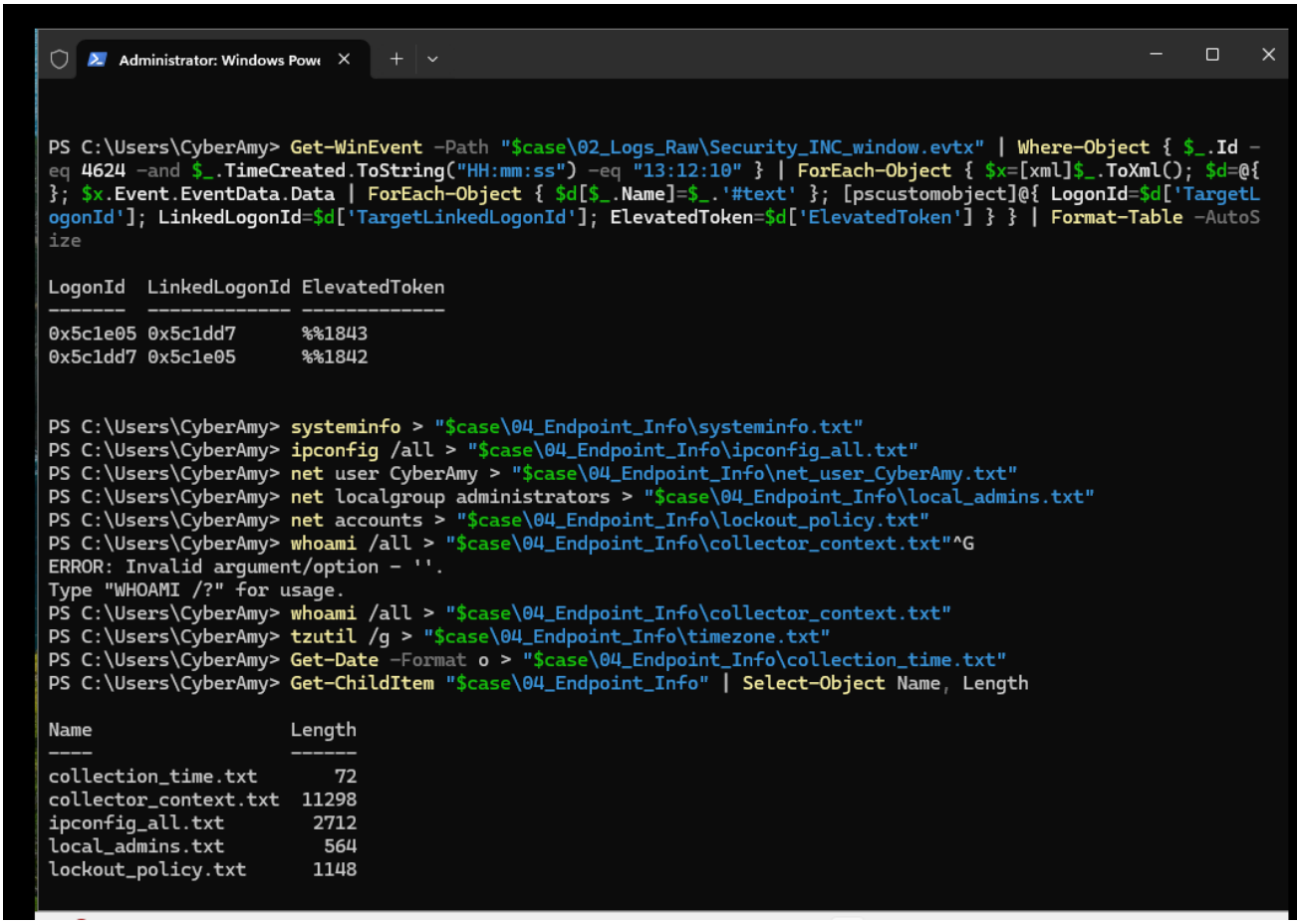


Figure 6. Linked-logon test result (saved in the case as SS06), followed by the endpoint captures.

What this screenshot shows:

- The table at the top has two rows. The session with LogonId 0x5c1e05 lists 0x5c1dd7 as its linked session, and 0x5c1dd7 lists 0x5c1e05: each points to the other.
- 0x5c1e05 has ElevatedToken %%1843 (No), the standard-user session. 0x5c1dd7 has %%1842 (Yes), the full administrator session.
- This is exactly the pattern of a UAC split token: one password entry, two linked sessions, one elevated and one not.
- The rest of the screenshot shows the endpoint information commands described in Section 7.

Both predictions held. This is one logon action recorded as two linked events, so the Day 25 conclusion of a single successful logon stands, now with proof. A split token only occurs for administrators, which Figure 8 confirms.

7. Endpoint Information Collected

Step 6: Capture the state of the endpoint

Each command's output was redirected (>) into its own text file, so the state of the machine at collection time is preserved alongside the logs:

```
systeminfo > "$case\04_Endpoint_Info\systeminfo.txt"
ipconfig /all > "$case\04_Endpoint_Info\ipconfig_all.txt"
net user CyberAmy > "$case\04_Endpoint_Info\net_user_CyberAmy.txt"
net localgroup administrators > "$case\04_Endpoint_Info\local_admins.txt"
net accounts > "$case\04_Endpoint_Info\lockout_policy.txt"
whoami /all > "$case\04_Endpoint_Info\collector_context.txt"
tzutil /g > "$case\04_Endpoint_Info\timezone.txt"
Get-Date -Format o > "$case\04_Endpoint_Info\collection_time.txt"
```

File	Purpose
systeminfo.txt	OS, build and hostname; identifies the endpoint
ipconfig_all.txt	Network identity at collection time
net_user_CyberAmy.txt	Affected account details, including last logon
local_admins.txt	Proves the account's privilege level
lockout_policy.txt	Lockout threshold of 10, not reached by 8 failures
collector_context.txt	Records the account doing the collecting
timezone.txt	Pins timestamps to a known time zone
collection_time.txt	Exact collection time in ISO 8601 format with UTC offset

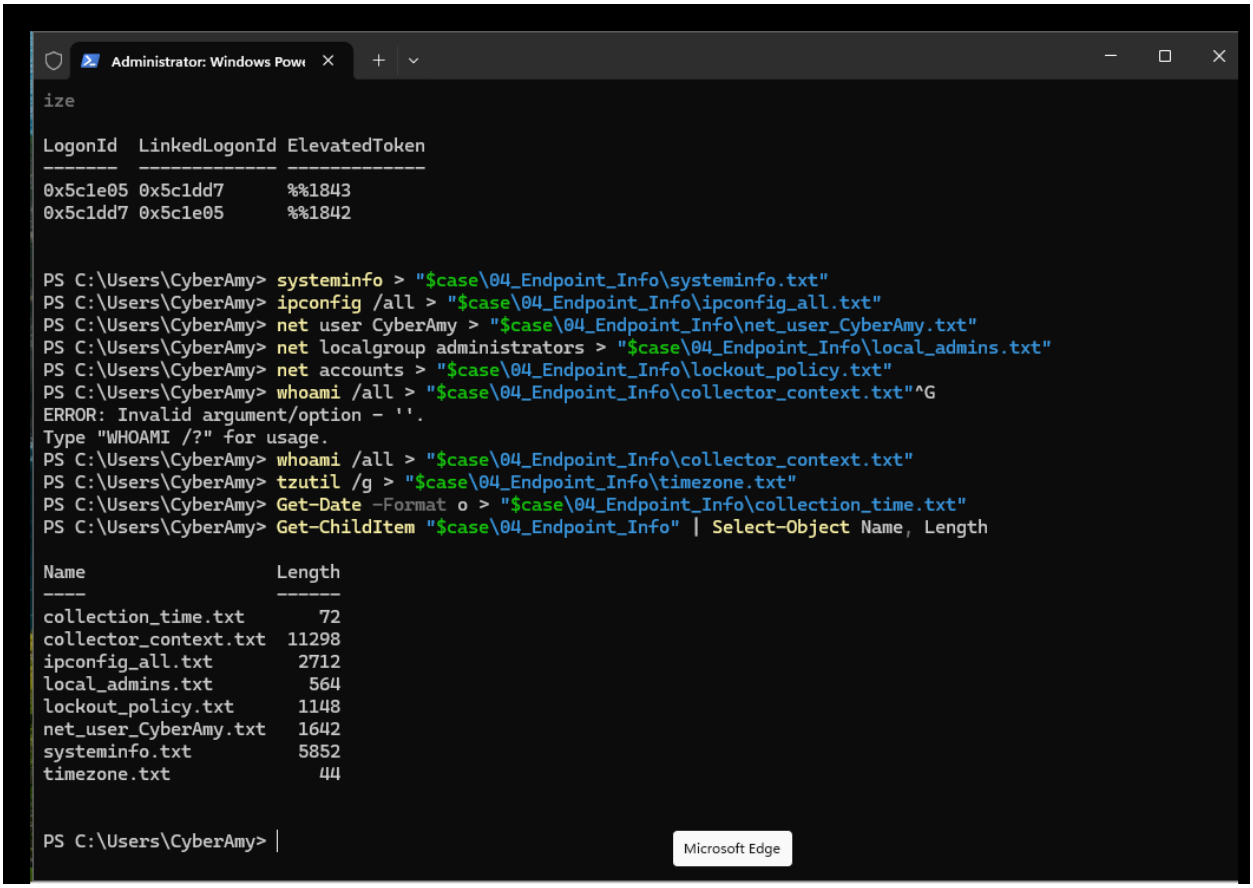


Figure 7. All eight endpoint commands and the resulting file list.

What this screenshot shows:

- The redirect commands print nothing when they succeed, so most lines show no output.
- The whoami /all line failed the first time with ERROR: Invalid argument/option, because the same hidden ^G character was attached during paste. The command was retyped on the next line and succeeded, overwriting the incomplete attempt.
- The final listing confirms all eight files exist with non-zero sizes; collector_context.txt at 11,298 bytes shows the retyped whoami /all captured its full output.

Step 7: Confirm the account's privilege level

```
Get-Content "$case\04_Endpoint_Info\local_admins.txt"
```

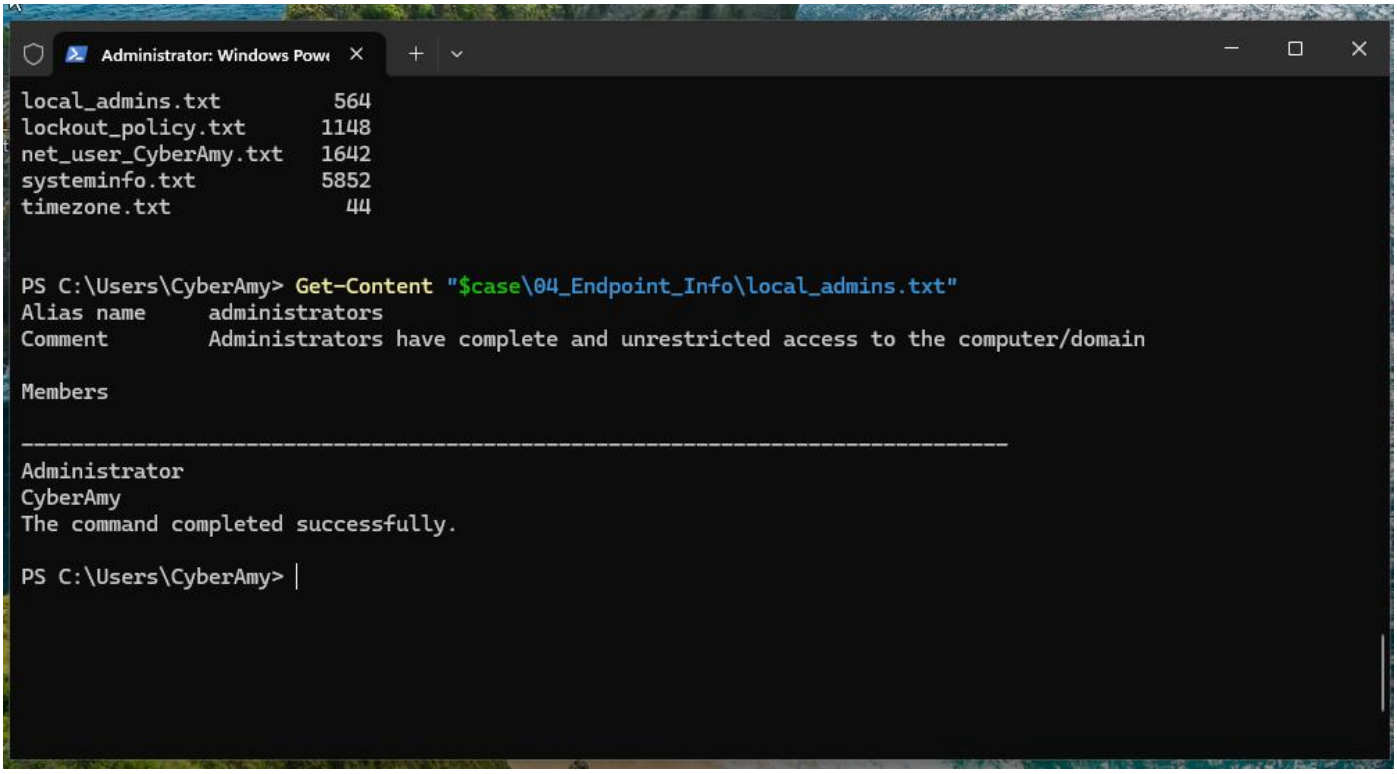


Figure 8. Members of the local Administrators group.

What this screenshot shows:

- Get-Content displays the saved file without changing it.
- Under Members, the group contains the built-in Administrator account and CyberAmy.
- This confirms CyberAmy is a local administrator, which is the precondition for the UAC split-token logon proven in Figure 6.

The collector context file also records that the evidence was collected while logged on as CyberAmy, the same account under investigation. That is acceptable in a lab where the analyst and the account owner are the same person, but in a real incident a separate analyst account must be used, and the chain of custody states this explicitly.

8. Screenshots and the Wazuh Folder

8.1 Contents of 01_Screenshots

Six screenshots were given numbered, descriptive names so the folder reads in order. They were captured on the host laptop and moved into the VM through Google Drive, which the chain of custody records. SS05 and SS06 appear above as Figures 5 and 6. SS01 to SS04 were captured during Day 25 and are reproduced here:

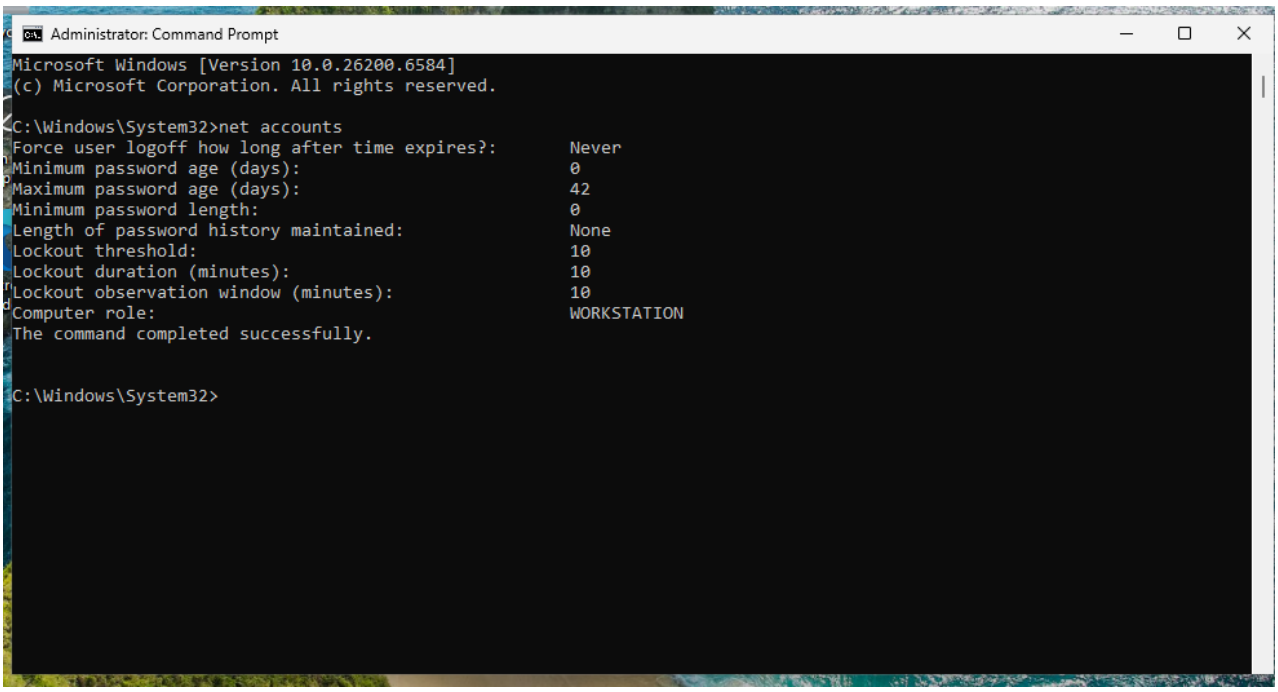


Figure 9. SS01_net_accounts_lockout_policy.png

What this screenshot shows:

- net accounts shows the machine's account lockout policy.
- Lockout threshold 10, Lockout duration 10 minutes, Lockout observation window 10 minutes: an account locks after 10 failed attempts within 10 minutes.
- The eight test failures stayed below this threshold, which is why the account was never locked.

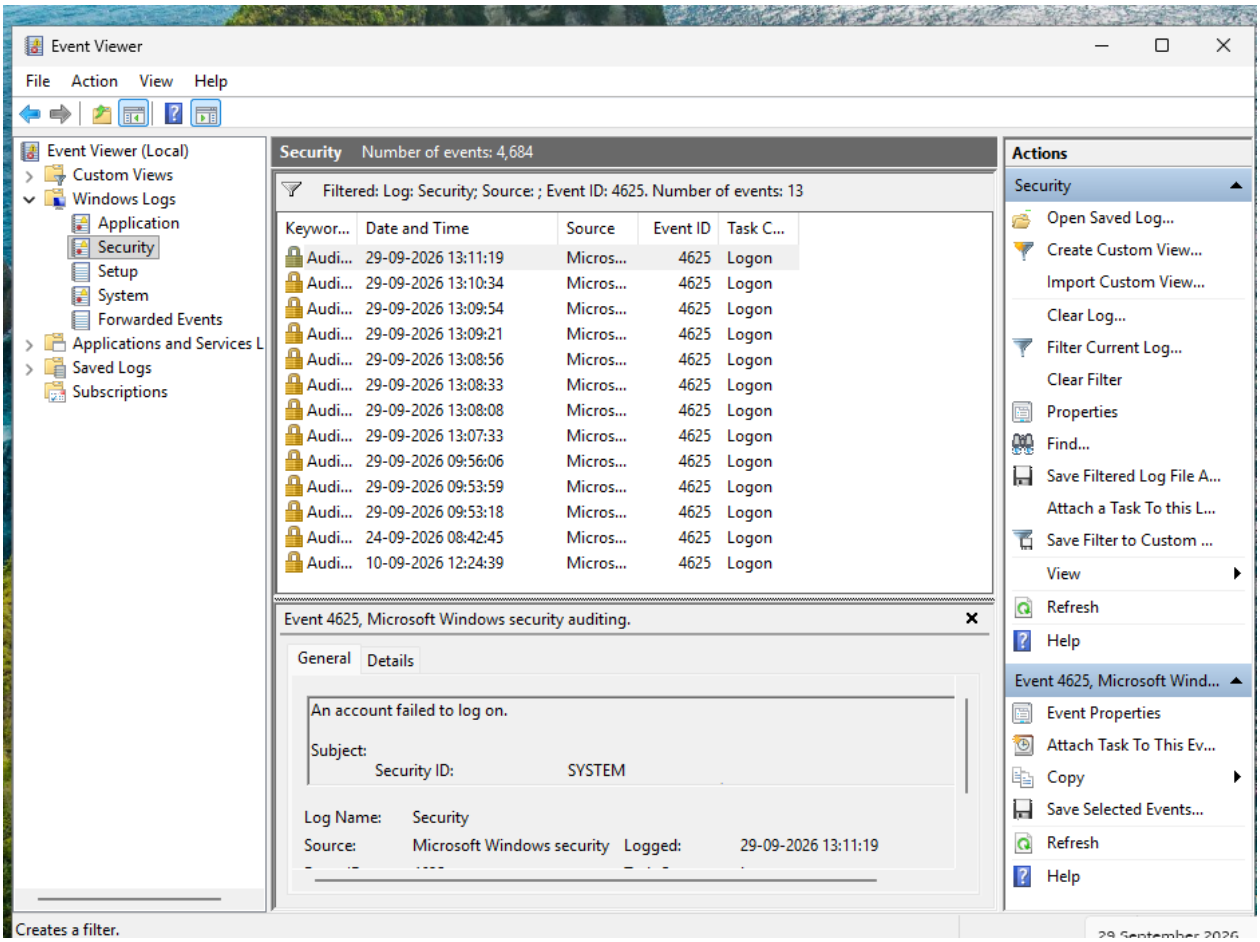


Figure 10. SS02_EventViewer_4625_filtered_list.png

What this screenshot shows:

- Event Viewer's Security log filtered to Event ID 4625 (failed logons).
- The eight entries from 13:07:33 to 13:11:19 on 29-09-2026 are the incident. Entries lower in the list are from earlier, unrelated sessions and were excluded by their timestamps.

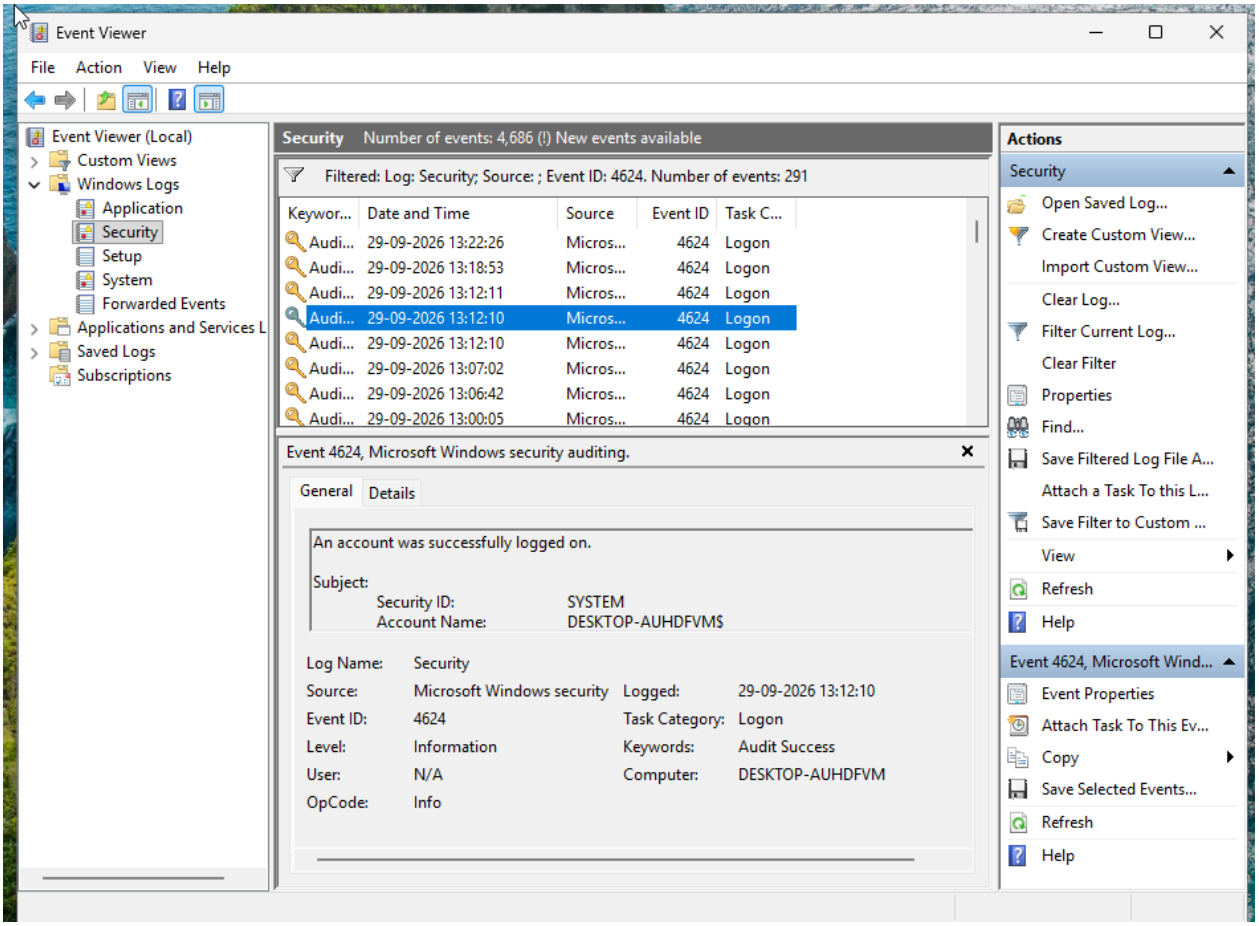


Figure 11. SS03_EventViewer_4624_1312_Subject.png

What this screenshot shows:

- The 13:12:10 successful logon (Event ID 4624), General tab, scrolled to the Subject section.
- Subject shows Security ID SYSTEM and Account Name DESKTOP-AUHDFVM$. A trailing $ marks a computer account: Subject identifies the process that requested the logon, not the person who logged on.
- Stopping at this section led to a misreading during Day 25, which is why the next screenshot matters.

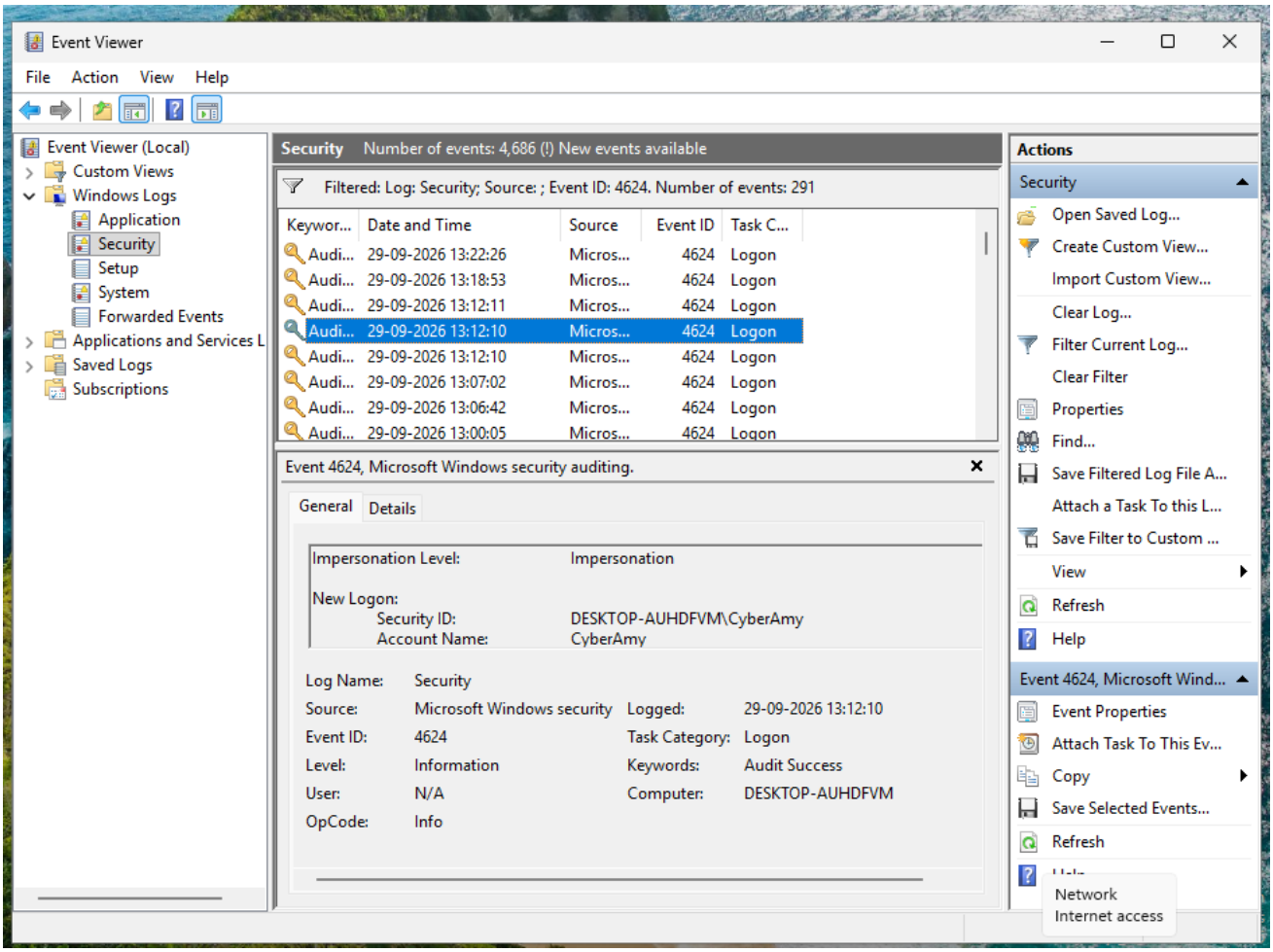


Figure 12. SS04_EventViewer_4624_1312_NewLogon.png

What this screenshot shows:

- The same 13:12:10 event, scrolled further down to the New Logon section.
- New Logon shows Security ID DESKTOP-AUHDFVM\CyberAmy and Account Name CyberAmy: this is the account that actually logged on.
- Together, Figures 11 and 12 show why an analyst must read the New Logon section of a 4624 event, not only the Subject section.

8.2 The Wazuh Status Note

05_Wazuh contains a status note instead of an alert. It states that no manager is deployed, names the rules that would fire (60122, level 5, for each failure; 60204, level 10, once 8 failures from the same source IP occur within 240 seconds), and records this incident's data against them: 8 failures in 226 seconds from 127.0.0.1. An empty folder invites the question of whether something was forgotten; a note answers it.

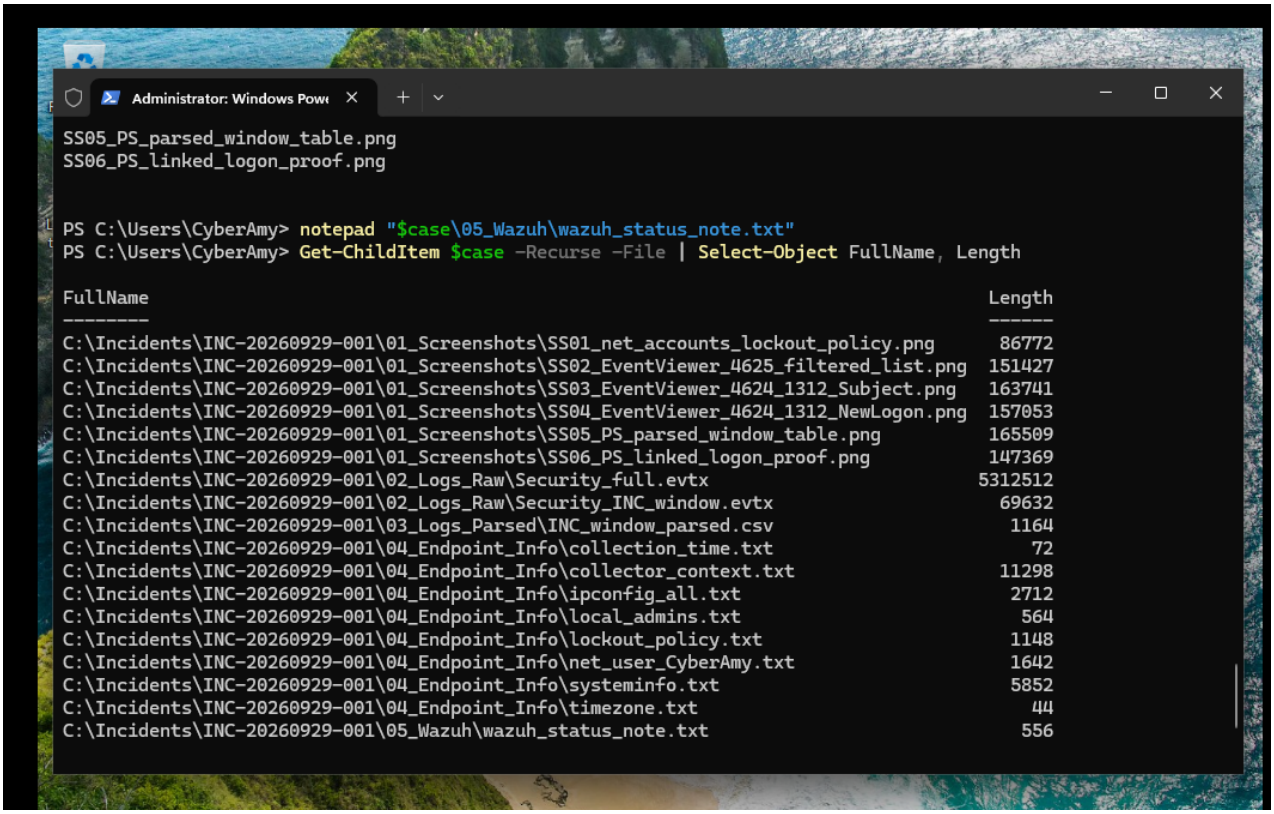


Figure 13. The Wazuh note opened in Notepad, then a recursive listing of all evidence files.

What this screenshot shows:

- The first command opened wazuh_status_note.txt in Notepad for writing.
- Get-ChildItem -Recurse -File then listed every file under the case folder: 6 screenshots, 2 raw logs, 1 parsed CSV, 8 endpoint files and the Wazuh note, 18 evidence files in total.
- This listing was the check that nothing was missing and nothing extra had crept in before the case notes were written.

9. Case Notes and Chain of Custody

Two documents in 00_Case_Notes make the package understandable without the collector present:

- case_summary.txt: incident ID, title, status, affected endpoint and user, source, an IST timeline, the analysis, and an index of every folder.
- chain_of_custody.txt: who collected the evidence and when, the method used for each type of evidence, every handling event during collection, and where the evidence is stored and how its integrity is protected.

Writing these notes exposed the third handling problem of the session:

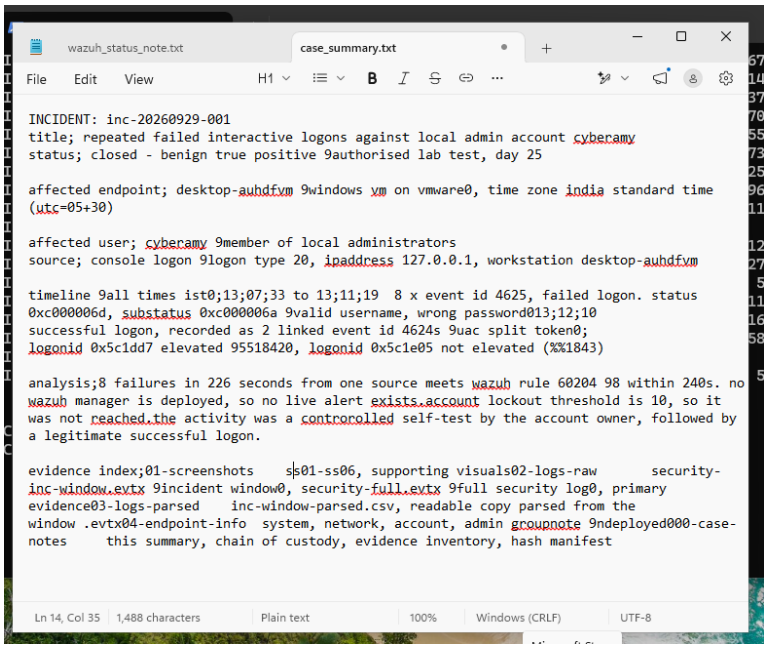


Figure 14. The case summary as it appeared after being pasted into Notepad inside the VM.

What this screenshot shows:

- Every character that needs the Shift key was lost: colons became semicolons, opening brackets became 9 and closing brackets became 0, and all capitals became lowercase. %%1842 appears as 95518420.
- Line breaks were also lost in the Evidence Index, and longer pastes were cut off part-way.
- A file like this cannot go into an evidence package, so it was discarded. The notes were written on the host laptop instead and moved into the VM through Google Drive.

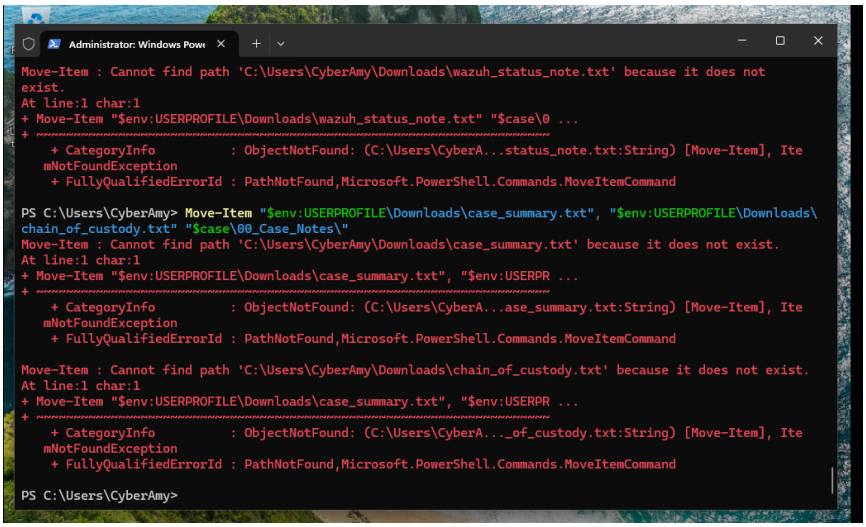


Figure 15. Move-Item errors while bringing the replacement notes into the case.

What this screenshot shows:

- Each Move-Item command reports Cannot find path ... because it does not exist, for all three note files in Downloads.
- The files were not in Downloads because they had already been saved directly into the case folders during download, as had happened earlier with the screenshots. Nothing was moved or lost.

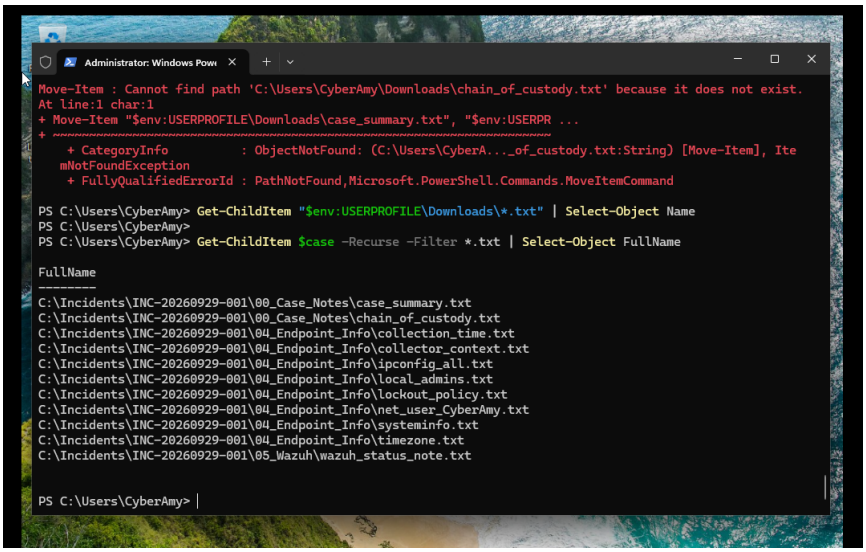


Figure 16. Locating the note files.

What this screenshot shows:

- The first command lists .txt files in Downloads and returns nothing, confirming they were never there.
- The second searches the whole case folder and shows case_summary.txt and chain_of_custody.txt in 00_Case_Notes and wazuh_status_note.txt in 05_Wazuh: each already in its correct place.

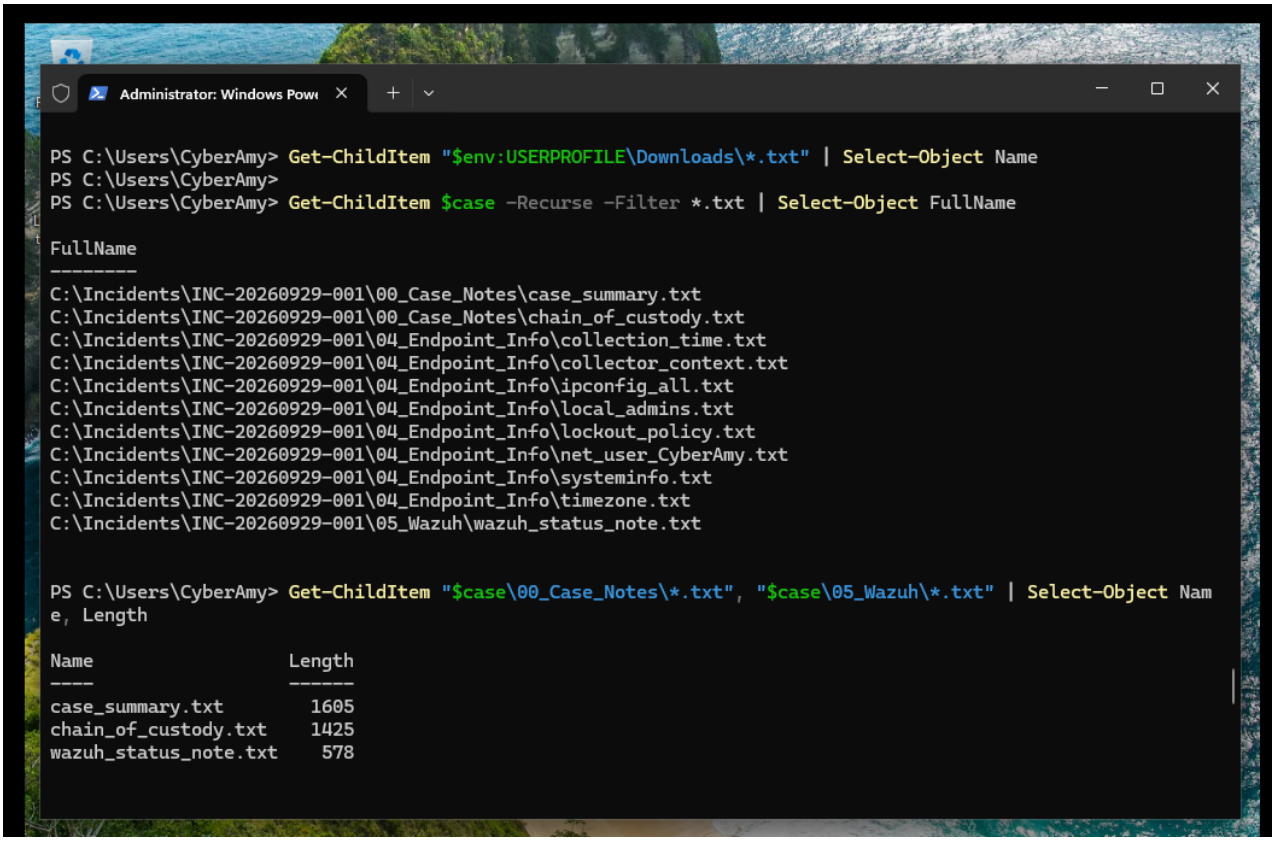


Figure 17. Confirming the clean versions by file size.

What this screenshot shows:

- A file's size is a quick way to confirm which version is present. The clean files written on the host were 1,605, 1,425 and 578 bytes.
- The listing shows exactly those sizes, so the garbled copies had been fully replaced before any hashing took place.

10. Sealing the Evidence

Hashing was deliberately the final step, because once a file's fingerprint is recorded, that file must never change again.

Step 8: **Write the evidence inventory**

```
Get-ChildItem $case -Recurse -File |
  Where-Object { $_.FullName -notlike "*\00_Case_Notes\*" } |
  Select-Object @{n='Item';e={$_.FullName.Replace("$case\","")}},
    @{n='LastWritten';e={$_.LastWriteTime.ToString("yyyy-MM-dd HH:mm:ss")}},
    Length |
  Export-Csv "$case\00_Case_Notes\evidence_inventory.csv" -NoTypeInformation
```

Step 9: **Hash every evidence file**

```
Get-ChildItem $case -Recurse -File |
  Where-Object { $_.FullName -notlike "*\00_Case_Notes\*" } |
  Get-FileHash -Algorithm SHA256 |
  Select-Object Hash, @{n='Item';e={$_.Path.Replace("$case\","")}} |
  Export-Csv "$case\00_Case_Notes\hash_manifest.csv" -NoTypeInformation
Import-Csv "$case\00_Case_Notes\hash_manifest.csv" | Format-Table -AutoSize
```

- Where-Object -notlike excludes 00_Case_Notes. Case notes are working documents that may still be edited, and a manifest cannot contain its own hash, because writing the hash into the file would change the file.
- Get-FileHash -Algorithm SHA256 calculates each file's fingerprint. SHA256 is also the cmdlet's default; naming it explicitly makes the choice visible in the record.
- The Select-Object expressions shorten each full path to a path relative to the case folder, so the inventory and manifest stay valid if the whole case is later copied elsewhere.

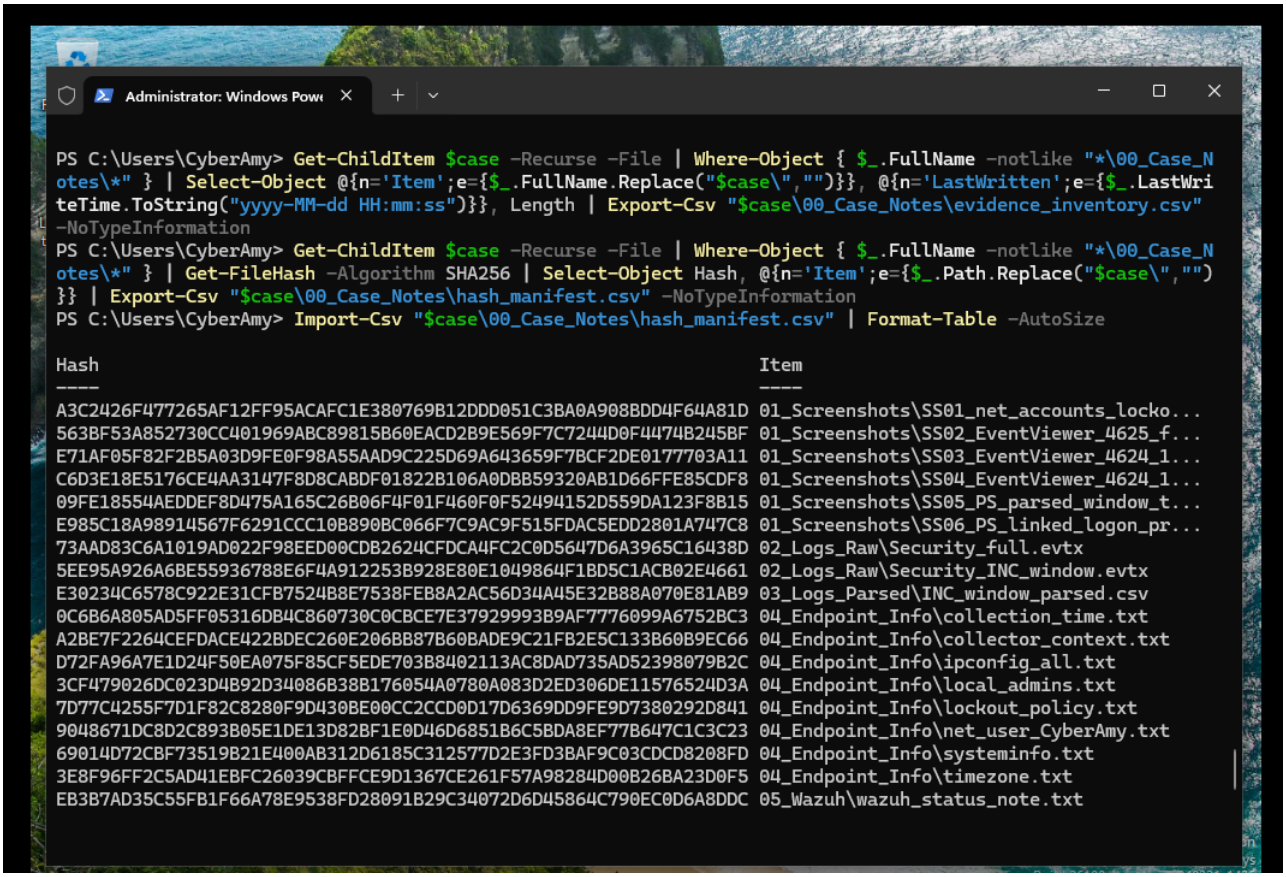


Figure 18. Inventory and hash commands, followed by the hash manifest.

What this screenshot shows:

- Both commands completed without errors; -NoTypeInformation was typed by hand this time and was accepted.
- Import-Csv reads hash_manifest.csv back and displays it: 18 rows, one for every evidence file in folders 01 to 05.
- Each Hash value is a 64-character SHA-256 fingerprint. Re-running Get-FileHash on any file later and getting the same value proves the file is unchanged. The manifest file in the case is the authoritative record of each value.

Step 10: Confirm the finished case

```
Get-ChildItem $case -Recurse -File |
  Select-Object @{n='Item';e={$_.FullName.Replace("$case\","")}}, Length |
  Format-Table -AutoSize
```

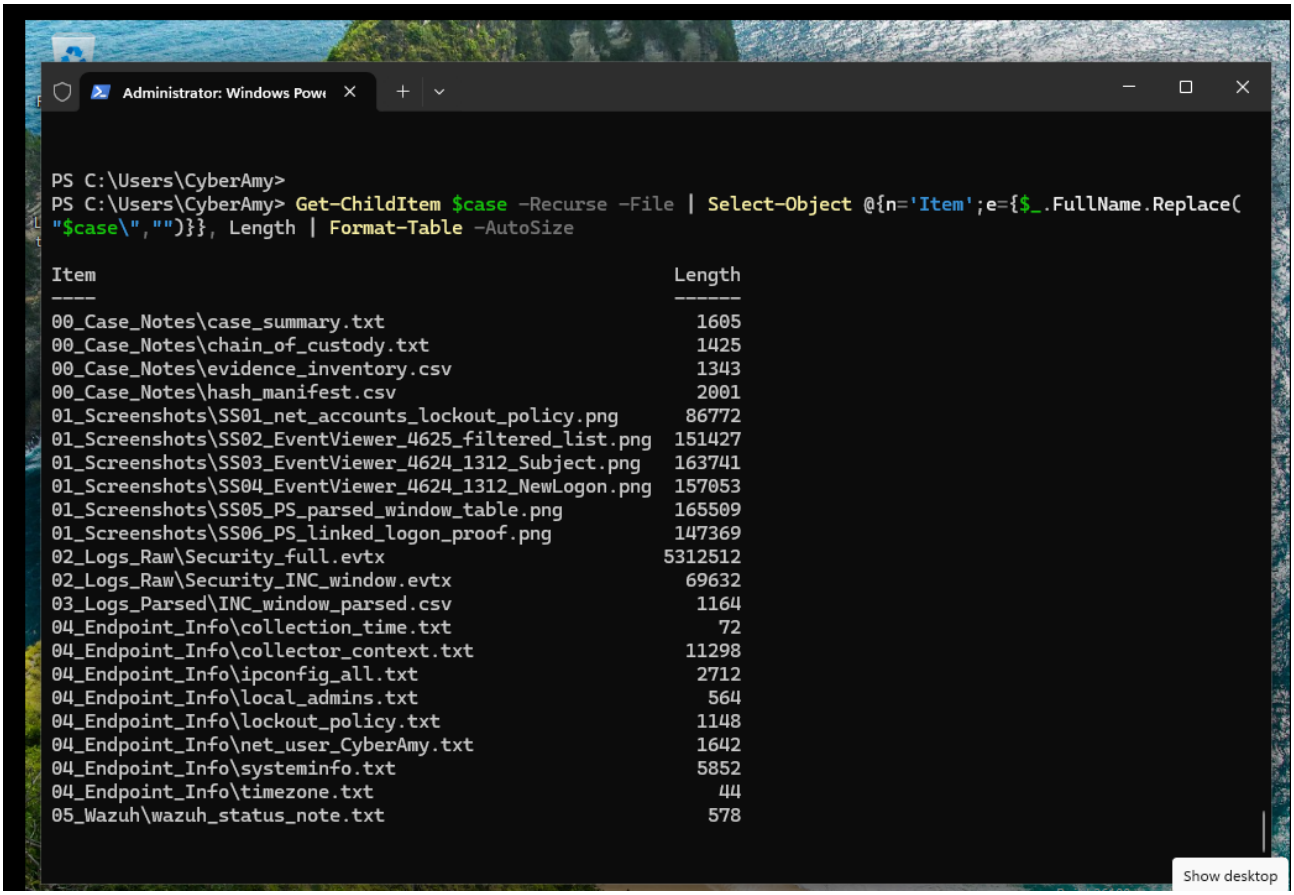


Figure 19. The complete, sealed case folder.

What this screenshot shows:

- 22 files in total: 4 case-note files in 00_Case_Notes (summary, chain of custody, inventory and manifest), plus the 18 hashed evidence files.
- The case-note sizes (1,605 and 1,425 bytes) match the clean versions verified in Figure 17.
- The evidence sizes match every earlier listing, for example Security_full.evtx at 5,312,512 bytes, showing nothing changed between collection and sealing.

11. Handling Issues During Collection

Three problems occurred during collection. Each was caught, corrected before the evidence was sealed, and recorded in the chain of custody, because an honest record of handling is what makes evidence defensible.

Issue	Effect	Resolution
Hidden ^G character appended when pasting commands (Figures 3 and 7)	Two commands rejected before running; nothing written	Short lines typed by hand; commands re-run successfully
Move-Item pulled unrelated PNG files from Downloads into 01_Screenshots	Stray files in the screenshots folder	Identified and moved back before hashing; folder left with exactly SS01 to SS06
Pasted text lost Shift-key characters and was truncated (Figure 14)	Unusable case notes	Notes written on the host, moved in via Google Drive, verified by size (Figure 17)

The practical lesson from the first issue: when an error names a parameter that is spelled correctly, suspect invisible characters and retype the end of the line.

12. Evidence Summary

Incident ID	INC-20260929-001
Affected Endpoint	DESKTOP-AUHDFVM, India Standard Time (UTC+05:30)
Affected User	CyberAmy (local administrator)
Timestamps	8 failures 13:07:33 to 13:11:19 IST; success 13:12:10 IST
Source Details	Logon Type 2 (Interactive), source IP 127.0.0.1, workstation DESKTOP-AUHDFVM
Failure Reason	Status 0xC000006D, SubStatus 0xC000006A (user name correct, password wrong)
Relevant Logs	Security_INC_window.evtx, Security_full.evtx, INC_window_parsed.csv
Screenshots	SS01 to SS06
Integrity	SHA-256 hashes of all 18 evidence files in hash_manifest.csv

Wazuh	No manager deployed; gap documented in 05_Wazuh
Case Status	Closed - benign true positive (authorized lab test)

13. Conclusion

Day 26 converted a single suspicious pattern into a complete, sealed evidence package. Every element the task asked for was collected from the real endpoint: the affected endpoint and user, timestamps normalized between local time and UTC, source details, raw and parsed logs, and screenshots, organized in a consistent numbered folder structure with case notes, a chain of custody record, and SHA-256 hashes.

Collecting the evidence properly also produced better analysis than Day 25. Re-reading the exported log showed that the attempts used a valid user name, that they originated at the local console, and that the double logon event was a UAC split token, each proven from the evidence rather than assumed. The one missing element, a Wazuh alert, is absent only because no manager is deployed in this lab, and that gap is documented inside the case itself so any future reviewer understands why.

References

- Microsoft Learn: 4624(S) An account was successfully logged on (logon types, Linked Logon ID, Elevated Token, source address).
- Microsoft Learn: wevtutil command reference (epl / export-log and the /q XPath option).
- Microsoft Learn: Get-FileHash (default SHA256 algorithm).
- IETF RFC 3227: Guidelines for Evidence Collection and Archiving (order of volatility).
- Ultimate Windows Security: Event ID 4624 (UAC split token and linked logon sessions).
- Wazuh ruleset, 0580-win-security_rules.xml (rules 60122 and 60204).